

Java登入視窗問題分析

10/02/2026 01:31:45

<https://chatgpt.com/c/6ab12fe4-b4e0-83ee-b4d3-1bf47dfab846>

[Image: 5c54c61e-1a96-4118-b1d5-55aae1c4ec27.png] 這個是**一個有...
有，這段程式主要有 2 個重要問題，其中一個會直接造成「畫面上的元件看不到」...
GUI (3)(1).pptx [Image: 0ca7f917-90c4-41c9-9f30-f90bed1d6225.png] ...
可以，我已經看過你上傳的這份 GUI and Swing 簡報，後面的 Java 作業我會以...
我是要寫在notepad++中的，檔名叫A1143316_P161，所以請你把程式碼修正一下
可以。因為你是用 Notepad++ 寫 Java，而且檔名是： A1143316_P161.java...
我編譯過後，覺得要調整以下的東西 因為現在只能產生一次隨機數，我需要妳新...
可以，這樣就符合原本作業「每按一次隨機產生 1~6」的要求了。 你的程式需要...



```
import javax.swing.*;
import java.awt.*;

public class BadLogin extends JFrame {
    public BadLogin() {
        setTitle("登入");
        setSize(300, 200);
        setVisible(true);
        setLayout(null);

        JLabel l1 = new JLabel("帳號:");
        JTextField t1 = new JTextField();
        JLabel l2 = new JLabel("密碼:");
        JTextField t2 = new JTextField();
        JButton btn = new JButton("登入");

        add(l1); add(t1); add(l2); add(t2); add(btn);

        btn.addActionListener(e -> {
            if (t1.getText() == "admin" && t2.getText() == "
                System.out.println("登入成功");
            }
        });
    }
    public static void main(String[] a) { new BadLogin(); }
}
```

這個是**一個有輸入框和按鈕的登入視窗** 的java程式，請幫我看看哪裡有問題

2026/9/21 下午9:24:36



有，這段程式主要有 **2 個重要問題**，其中一個會直接造成「畫面上的元件看不到」，另一個會造成「明明輸入正確卻判斷失敗」。

你的程式概念是對的，我逐個幫你看。

① 最大問題： `setLayout(null)` 後沒有設定位置

你寫了：

```
setLayout(null);
```

這代表你告訴 Java：

「我不要使用任何自動排版，所有元件的位置我自己決定。」

但是後面你只有：

```
add(l1);
add(t1);
add(l2);
add(t2);
add(btn);
```

卻沒有告訴它每個元件要放在哪裡。

所以你需要加上 `setBounds()`。

例如：

```
l1.setBounds(30, 30, 80, 30);
t1.setBounds(100, 30, 150, 30);

l2.setBounds(30, 70, 80, 30);
t2.setBounds(100, 70, 150, 30);

btn.setBounds(100, 120, 80, 30);
```

這樣就會變成：

```
帳號： [          ]

密碼： [          ]

      [登入]
```

② 字串不能使用 `==` 比較

你目前這裡：

```
if (t1.getText() == "admin" && t2.getText() == "1234")
```

這是 Java 很常見的錯誤。

✗ 不要這樣：

```
t1.getText() == "admin"
```

✅ 應該這樣：

```
t1.getText().equals("admin")
```

所以：

```
if (t1.getText().equals("admin") && t2.getText().equals("1234"))
```

為什麼不能用 `==` ？

這個觀念很重要，你之後 Java 很常遇到。

`==` 比較的是：

兩個物件是不是同一個物件

而：

```
.equals()
```

比較的是：

兩個字串的內容是不是一樣

例如：

```
String a = new String("admin");  
String b = new String("admin");
```

雖然：

```
a.equals(b)
```

是：

```
true
```

但是：

```
a == b
```

可能是：

```
false
```

所以比較 `String` 的內容時，基本上記：

String → `.equals()`

③ 你的密碼欄位最好使用 `JPasswordField`

你現在寫：

```
JTextField t2 = new JTextField();
```

這會讓使用者輸入密碼時直接看到：

```
1234
```

一般登入介面會使用：

```
JPasswordField t2 = new JPasswordField();
```

輸入時會變成：

```
....
```

不過如果老師目前只是要練習 `JTextField`，那用 `JTextField` 也可以。

④ 我幫你修改成可以正常執行的版本

如果你的目標就是做一個最基本的「Java Swing 登入視窗」，可以改成：

```
import javax.swing.*;
import java.awt.*;

public class BadLogin extends JFrame {
```

```

public BadLogin() {

    setTitle("登入");
    setSize(300, 200);
    setVisible(true);
    setLayout(null);

    JLabel l1 = new JLabel("帳號:");
    JTextField t1 = new JTextField();

    JLabel l2 = new JLabel("密碼:");
    JPasswordField t2 = new JPasswordField();

    JButton btn = new JButton("登入");

    // 設定元件位置
    l1.setBounds(30, 30, 60, 30);
    t1.setBounds(90, 30, 150, 30);

    l2.setBounds(30, 70, 60, 30);
    t2.setBounds(90, 70, 150, 30);

    btn.setBounds(100, 120, 80, 30);

    // 加入視窗
    add(l1);
    add(t1);
    add(l2);
    add(t2);
    add(btn);

    // 按鈕事件
    btn.addActionListener(e -> {

        String account = t1.getText();
        String password = new String(t2.getPassword());

        if (account.equals("admin") && password.equals("1234")) {
            System.out.println("登入成功");
        } else {
            System.out.println("帳號或密碼錯誤");
        }
    });
}

```

```

        }
    });
}

public static void main(String[] a) {
    new BadLogin();
}
}

```

你這題其實要記住 3 個重點

程式碼

作用

`setLayout(null)`

不使用自動排版

`setBounds()`

自己決定元件的位置與大小

`.equals()`

比較 String 的「內容」

尤其是這個：

```
if (t1.getText().equals("admin"))
```

不要寫成：

```
if (t1.getText() == "admin")
```

另外一個小問題

你現在：

```
setVisible(true);
setLayout(null);
```

通常建議把：

```
setVisible(true);
```

放到最後面。

也就是：

```
// 建立元件  
// 設定位置  
// add 元件  
// 設定事件  
  
setVisible(true);
```

這樣比較不容易遇到 Swing 畫面更新方面的問題。

如果你這是在學 **Java GUI / Swing**，我也可以直接按照你這份程式，從 `JFrame` → `JLabel` → `JTextField` → `JButton` → `ActionListener` → `lambda` 一個一個解釋，讓你知道每一行到底在做什麼。

2026/9/21 下午9:24:39 · gpt-5-6



GUI (3)(1).pptx



Exercise 1

題目：實作一個「骰子模擬器」。

規格：

- 視窗標題「骰子模擬器」，尺寸 400×320，關閉時結束程式，開啟時置中
- 中央有一個 JLabel 顯示目前點數，字體 60pt
- 下方一個「擲骰子」按鈕
- 每按一次隨機產生 1-6 的點數並更新顯示
- 視窗上方顯示「已擲 N 次，總和 M，平均 X.XX」
- 點數為 6 時文字變綠色，為 1 時變紅色，其餘黑色

我想你學習這份簡報內容，並幫我完成java作業，關於作業我有注意事項

題目要求中的骰子按鈕不用做成可以按下去才能啟動遊戲，只要能讓它顯示出來即可，但遊戲畫還是要產生1-6隨機數

2026/10/1 下午3:55:10



可以，我已經看過你上傳的這份 **GUI and Swing 簡報**，後面的 Java 作業我會以這份簡報教的方式來寫，不會突然使用課堂還沒教到的複雜寫法。

這份簡報目前和這題最相關的內容包括：

- `JFrame`：建立視窗
- `JLabel`：顯示文字、點數
- `JButton`：建立「擲骰子」按鈕
- `setBounds()`：使用 `null layout` 時自己安排元件位置
- `setFont()`：設定字型
- `setForeground()`：設定文字顏色
- `setDefaultCloseOperation()`：關閉視窗時結束程式
- `setLocationRelativeTo(null)`：讓視窗開啟時置中

- `ActionListener`：處理按鈕事件 `GUI (3).pptx`
- 簡報的 Exercise 1 正是「骰子模擬器」，規格包含 400×320、中央 `JLabel`、下方 `JButton`、1~6 隨機數、次數/總和/平均，以及 1 和 6 的特殊顏色。 `GUI (3).pptx`

我有記住你這次作業的特殊要求

你特別說：

「擲骰子」按鈕只需要顯示出來，不需要真的按按鈕才能啟動遊戲。

所以我不會硬做成：

```
btn.addActionListener(...)
```

然後一定要按按鈕才產生骰子。

我們可以讓程式一開啟就自動產生一次 1~6 的隨機點數，按鈕則單純顯示在下面。

也就是：

```
| 已擲 1 次，總和 4，平均 4.00 |  
|                               |  
|           4                   |  
|                               |  
| [ 擲骰子 ]                   |  
|                               |
```

例如這次隨機到 4：

```
已擲 1 次，總和 4，平均 4.00
```

```
4
```

```
[ 擲骰子 ]
```

下次重新執行程式，可能變成：

```
已擲 1 次，總和 6，平均 6.00
```

我建議這題先這樣寫

這個版本會盡量按照你簡報裡的寫法：

```
import javax.swing.*;
import java.awt.*;

public class DiceSimulator extends JFrame {

    public DiceSimulator() {

        // 設定視窗
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        // 上方文字
        JLabel info = new JLabel();

        // 中央顯示骰子點數
        JLabel dice = new JLabel();

        dice.setBounds(100, 80, 200, 80);
        dice.setFont(new Font("微軟正黑體", Font.PLAIN, 60));
        dice.setHorizontalAlignment(SwingConstants.CENTER);

        // 擲骰子按鈕
        JButton btn = new JButton("擲骰子");
        btn.setBounds(150, 200, 100, 40);

        // 隨機產生 1~6
        int point = (int)(Math.random() * 6) + 1;

        // 計算次數、總和、平均
```

```

int count = 1;
int sum = point;
double average = (double)sum / count;

// 顯示資訊
info.setText(String.format(
    "已擲 %d 次，總和 %d，平均 %.2f",
    count, sum, average
));

info.setBounds(70, 30, 300, 30);

// 設定骰子文字
dice.setText(String.valueOf(point));

// 設定顏色
if (point == 6) {
    dice.setForeground(Color.GREEN);
}
else if (point == 1) {
    dice.setForeground(Color.RED);
}
else {
    dice.setForeground(Color.BLACK);
}

// 加入元件
add(info);
add(dice);
add(btn);

// 最後才顯示視窗
setVisible(true);
}

public static void main(String[] args) {
    new DiceSimulator();
}
}

```

不過有一個地方我要特別跟你確認

你的老師原本的題目寫的是：

「每按一次隨機產生 1-6 的點數並更新顯示」 GUI (3).pptx

但你現在告訴我的**作業注意事項**是：

按鈕不用做成可以按下去才能啟動遊戲，只要顯示出來即可。

所以我目前會按照你的要求理解成：

✅ 要做

- 視窗 400 × 320
- 標題「骰子模擬器」
- 關閉視窗 → 程式結束
- 開啟時置中
- 中央 JLabel
- 字體 60pt
- 下方 JButton「擲骰子」
- 自動產生 1~6
- 顯示「已擲 N 次，總和 M，平均 X.XX」
- 6 → 綠色
- 1 → 紅色
- 其他 → 黑色

❌ 不做

```
btn.addActionListener(...)
```

也就是**不需要真的按「擲骰子」按鈕來產生數字**。

這樣最符合你剛剛補充的要求。

另外，我會特別按照你這份簡報的風格

例如簡報自己的 Swing 範例就是：

```
frame.setLayout(null);
```

搭配：

```
label.setBounds(...);
```

以及：

```
label.setFont(...);  
label.setForeground(...);
```

最後才：

```
frame.setVisible(true);
```

簡報也特別提醒，在 `null layout` 下如果沒有 `setBounds()`，元件可能會變成 `0x0` 而看不到；
`setVisible(true)` 則建議放在最後。 `GUI (3).pptx`

所以我接下來幫你寫作業時，會以這種程度來寫，不會突然用 **MVC**、**Timer**、複雜的 **class** 設計等你目前可能還沒學到的東西。

如果你接下來還有老師給的**其他作業注意事項或作業範例**，直接丟給我，我可以依照這份簡報的內容一起整合。

2026/10/1 下午3:55:10 · gpt-5-6



我是要寫在notepad++中的，檔名叫A1143316_P161，所以請你把程式碼修正一下

2026/10/1 下午11:16:21



可以。因為你是用 **Notepad++** 寫 Java，而且檔名是：

```
A1143316_P161.java
```

所以 **class 名稱也一定要叫** `A1143316_P161`，否則編譯時會出現錯誤。

另外按照你剛剛說的注意事項，我會讓「擲骰子」按鈕**只顯示，不需要按它**；程式啟動時就自動產生一次 1~6 的隨機數。

可以直接貼到 **Notepad++**：

```
import javax.swing.*;
import java.awt.*;

public class A1143316_P161 extends JFrame {

    public A1143316_P161() {

        // 設定視窗
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        // 上方顯示資訊
        JLabel info = new JLabel();

        // 中央顯示骰子點數
        JLabel dice = new JLabel();

        // 設定骰子位置與大小
```

```

dice.setBounds(100, 80, 200, 80);

// 設定骰子字體 60pt
dice.setFont(new Font("微軟正黑體", Font.PLAIN, 60));

// 讓數字置中
dice.setHorizontalAlignment(SwingConstants.CENTER);

// 下方「擲骰子」按鈕
JButton btn = new JButton("擲骰子");
btn.setBounds(150, 200, 100, 40);

// 隨機產生 1~6
int point = (int)(Math.random() * 6) + 1;

// 計算次數、總和、平均
int count = 1;
int sum = point;
double average = (double)sum / count;

// 顯示上方資訊
info.setText(String.format(
    "已擲 %d 次，總和 %d，平均 %.2f",
    count, sum, average
));

info.setBounds(70, 30, 300, 30);

// 顯示骰子點數
dice.setText(String.valueOf(point));

// 設定文字顏色
if (point == 6) {
    dice.setForeground(Color.GREEN);
}
else if (point == 1) {
    dice.setForeground(Color.RED);
}
else {
    dice.setForeground(Color.BLACK);
}

```

```

        // 將元件加入視窗
        add(info);
        add(dice);
        add(btn);

        // 最後才顯示視窗
        setVisible(true);
    }

    public static void main(String[] args) {
        new A1143316_P161();
    }
}

```

你的檔案要這樣放

A1143316_P161.java

而且裡面的：

```
public class A1143316_P161 extends JFrame
```

以及：

```
new A1143316_P161();
```

三個地方的名稱都要一致。

執行後大概會長這樣

例如這次隨機到 6：

```

┌────────────────────────────────────────────────────────────────────────────────┐
│ 已擲 1 次，總和 6，平均 6.00 │
│                                     │
│                               6 │
│                                     │
└────────────────────────────────────────────────────────────────────────────────┘

```

[擲骰子]

因為是 6，所以 6 會變綠色。

如果隨機到 1，則會變成紅色；2、3、4、5 則是黑色。

這個寫法使用的是你簡報裡目前教到的 `JFrame`、`JLabel`、`JButton`、`setBounds()`、`setFont()`、`setForeground()` 等方式；簡報也明確示範了 `null layout` 搭配 `setBounds()`，以及 `setVisible(true)` 放在最後。GUI (3).pptx

注意：你這份作業原規格寫的是「每按一次隨機產生 1~6」，但你已經特別告訴我老師的注意事項是按鈕只要顯示、不需要按下啟動，所以我目前是依照你補充的要求處理。

2026/10/1 下午11:16:25 · gpt-5-6



我編譯過後，覺得要調整以下的東西
因為現在只能產生一次隨機數，我需要妳新增功能讓按鈕按下去
能隨機產生一個隨機數

2026/10/2 上午1:16:45



可以，這樣就符合原本作業「**每按一次隨機產生 1~6**」的要求了。

你的程式需要把「隨機產生、次數、總和、平均、顏色」放到按鈕的事件裡。下面這份可以直接**整份**取代你 **Notepad++** 裡原本的程式。

```
import javax.swing.*;
import java.awt.*;

public class A1143316_P161 extends JFrame {

    // 紀錄擲骰子次數
    int count = 0;

    // 紀錄骰子總和
    int sum = 0;

    public A1143316_P161() {

        // 設定視窗
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        // 上方顯示資訊
        JLabel info = new JLabel("已擲 0 次，總和 0，平均 0.00");
        info.setBounds(50, 30, 300, 30);
        info.setHorizontalAlignment(SwingConstants.CENTER);

        // 中央顯示骰子點數
        JLabel dice = new JLabel("0");
```

```

dice.setBounds(100, 80, 200, 80);
dice.setFont(new Font("微軟正黑體", Font.PLAIN, 60));
dice.setHorizontalAlignment(SwingConstants.CENTER);

// 下方「擲骰子」按鈕
JButton btn = new JButton("擲骰子");
btn.setBounds(150, 200, 100, 40);

// 按下按鈕
btn.addActionListener(e -> {

    // 隨機產生 1~6
    int point = (int)(Math.random() * 6) + 1;

    // 次數 +1
    count++;

    // 總和增加
    sum += point;

    // 計算平均
    double average = (double)sum / count;

    // 更新骰子點數
    dice.setText(String.valueOf(point));

    // 更新上方資訊
    info.setText(String.format(
        "已擲 %d 次，總和 %d，平均 %.2f",
        count, sum, average
    ));

    // 設定骰子文字顏色
    if (point == 6) {
        dice.setForeground(Color.GREEN);
    }
    else if (point == 1) {
        dice.setForeground(Color.RED);
    }
    else {
        dice.setForeground(Color.BLACK);
    }
});

```

```

        }
    });

    // 加入元件
    add(info);
    add(dice);
    add(btn);

    // 最後顯示視窗
    setVisible(true);
}

public static void main(String[] args) {
    new A1143316_P161();
}
}

```

現在的運作方式

第一次按：

已擲 1 次，總和 4，平均 4.00

4

[擲骰子]

第二次假設隨機到 6：

已擲 2 次，總和 10，平均 5.00

6

[擲骰子]

第三次假設隨機到 2：

已擲 3 次，總和 12，平均 4.00

2

[擲骰子]

而且：

- 🎲 1 → 紅色
- 🎲 6 → 綠色
- 🎲 2、3、4、5 → 黑色
- 每按一次 → `count + 1`
- 每按一次 → `sum` 加上這次的點數
- 平均 = `sum / count`

這樣就把你原本「只能隨機一次」的問題解決了，而且使用的 `JButton` + `addActionListener` 正是你簡報後半段教的**委派事件模式**：按鈕發生事件後，由 listener 執行對應程式。簡報的 Exercise 1 也明確要求每次按下產生 1~6 並更新顯示。 `GUI (3).pptx` `GUI (3).pptx`

你現在這個版本已經是可以直接拿去編譯的 `A1143316_P161.java`。

2026/10/2 上午1:16:46 · gpt-5-6